C89 Mini Ray Tracer — Coding Benchmark

Task

Implement, compile, and run a small standalone ray tracer in strict ANSI C89. Render two spheres side by side, one made of clear glass and one made of polished chrome, resting on an infinite black-and-white checkerboard plane. Use classic three-point lighting, a red-to-orange gradient sky, and distance fog that blends the ground into the sky.

The final image must be a **1280 × 720 bitmap saved as** `out.tga` in the current working directory.

You must complete the implementation and produce the image using the local `cc` compiler. Providing source code or build instructions alone does not complete the task.

Deliverables and implementation constraints

- Put the entire implementation in **one source file,** `main.c`. Standard C headers are allowed; do not create additional project `.c` or `.h` files.
- Use strict C89, including C89-compatible comments, declarations, and standard library calls. Do not use C99 features, compiler extensions, or C++.
- Use only the C standard library and the system math library. No external dependencies, graphics APIs, image libraries, downloaded assets, or external rendering tools.
- Do not create a Makefile, CMake configuration, build script, or package configuration.
- Perform all ray tracing and image encoding in the program itself, on the CPU.
- The executable must render the prescribed scene when run without arguments. It must require no input files, network access, or user interaction.
- Use deterministic sampling so repeated runs of the same executable produce the same image.
- Keep the source readable, with functions for vector math, intersections, materials, shading, the sky, fog, and TGA output. Briefly comment the non-obvious math.

Required generated artifacts: the `raytracer` executable and `out.tga`, alongside `main.c`.

Coordinate system and composition

Use a right-handed world with **Y pointing up**. All dimensions below are in arbitrary world units.

Element	Required configuration
Ground	Infinite plane at `y = 0`
Glass sphere	Center `(-1.25, 1.0, 0.0)`, radius `1.0`
Chrome sphere	Center `(1.25, 1.0, 0.0)`, radius `1.0`
Camera	Position `(0.0, 2.6, 6.5)`, looking at `(0.0, 1.0, 0.0)`
Camera up	`(0.0, 1.0, 0.0)`
Projection	Perspective, vertical field of view 45 degrees, aspect ratio `1280 / 720`

The glass sphere must appear on the left and the chrome sphere on the right. Both spheres must be fully visible, visibly rest on the plane, and have a small gap between them. The frame must show foreground checker tiles and the distant ground fading into the orange horizon.

Infinite checkerboard

- Compute the pattern procedurally from the world-space X and Z coordinates of each plane intersection.
- Each square measures `1.0 × 1.0` world units.
- Alternate black `(0.0, 0.0, 0.0)` and white `(1.0, 1.0, 1.0)` diffuse albedos.
- Use `floor` or equivalent correct logic for negative coordinates. The pattern must remain continuous across the world origin.
- Model an actual infinite plane: there must be no finite floor edge, surrounding empty strip, or arbitrary far clipping boundary.
- The plane must receive illumination and shadows cast by the spheres. It must also appear in their reflections and through the glass.

Glass sphere

- Clear, colorless dielectric with an index of refraction of **1.5**, surrounded by air with index **1.0**.
- Implement both reflection and refraction, with Fresnel weighting using Schlick's approximation or the exact dielectric equations.
- Correctly distinguish entering and exiting rays, orient the interface normal, and use the appropriate index ratio for Snell's law.
- Handle total internal reflection by reflecting all the energy assigned to the dielectric interface.
- Refracted rays must traverse the sphere and exit through its second interface. The checkerboard behind or beneath it must be visibly distorted.
- Preserve strong grazing-angle reflections and visible light highlights. Do not approximate glass with constant alpha blending or a translucent diffuse material.

Chrome sphere

- Polished, neutral silver metal with strong mirror-like reflections and sharp light highlights.
- Use a reflection weight of approximately `0.9–0.98`, with little or no diffuse contribution.
- Reflect the checkerboard, gradient sky, and other sphere where geometrically visible.
- It must read as reflective chrome rather than a uniformly gray or diffusely shaded ball.

Classic three-point lighting

Use three white point lights with these positions and relative strengths:

Light	Position	Relative strength	Purpose
Key	`(-3.5, 6.0, 4.5)`	`1.0`	Main light, above and to the camera's left
Fill	`(4.0, 3.0, 3.0)`	`0.35`	Softer illumination from the camera's right
Back / rim	`(0.5, 5.0, -4.0)`	`0.7`	Rear highlights that separate the spheres from the background

For simplicity, use these strengths without distance attenuation. Compute diffuse lighting for the ground and suitable specular highlights for the spheres. Analytic specular highlights are acceptable for the point lights. A small ambient contribution is allowed, but it must not flatten the lighting.

Trace shadow rays toward each light, stopping at the light's distance. Chrome blocks direct light. Glass must transmit at least some shadow-ray light; a simple tinted or neutral attenuation approximation is sufficient. Hard shadows are acceptable. Physically accurate caustics, area lights, and global illumination are not required.

Gradient sky

Evaluate the sky procedurally from the **normalized ray direction**, including for reflected and refracted rays that miss the scene.

Use the following linear RGB colors:

- Zenith and nadir: dark red, `(0.18, 0.01, 0.015)`.
- Horizon: orange, `(1.0, 0.25, 0.03)`.

For direction `d`, define:

```
t = pow(abs(d.y), 0.65)
sky(d) = (1 - t) * horizon_color + t * dark_red_color
```

Here `abs` denotes floating-point absolute value; use `fabs` in C. Clamp its input to `[0, 1]` if necessary for numerical robustness.

The gradient must be smooth and symmetric above and below the horizon. Although the infinite floor hides the nadir in the direct camera view, the environment function must still define it correctly.

Fog and the ground-to-sky transition

Use distance fog whose color matches the sky in the direction of the ray being shaded. For a finite ray segment that hits a surface at distance `distance`, use:

```
transmittance = exp(-0.035 * distance)
fog_color = sky(ray_direction)
result = transmittance * shaded_surface + (1 - transmittance) * fog_color
```

Apply this consistently to primary and secondary finite ray segments, once per segment. For a ray that misses all geometry, return the sky directly.

The distant checkerboard must gradually lose contrast and blend into the orange horizon without a visible cutoff. Nearby tiles and the spheres must remain clear. Do not use a constant gray fog color or a separate flat horizon band.

Ray tracing and image quality

- Implement analytic ray–sphere and ray–plane intersections, selecting the nearest valid positive hit.
- Support rays starting inside a sphere by considering both quadratic roots.
- Normalize ray directions so intersection distances and fog use the same world units.
- Offset reflection, refraction, and shadow ray origins to the correct side of a surface with a small epsilon, such as `1e-4`, to prevent self-intersection artifacts.
- Support at least **8 secondary-ray bounces**, with a finite recursion limit and a deliberate fallback at that limit.
- Use at least **4 primary samples per pixel**, for example a deterministic `2 × 2` subpixel grid, and average them in linear color space.
- Compute lighting, reflection/refraction mixing, fog, and sample averaging in linear RGB. Apply a consistent exposure or tone mapping if needed, then clamp and gamma-encode to 8-bit output, using gamma `2.2` or the standard sRGB transfer function.
- Avoid NaNs, invalid square roots, divisions by nearly zero, unchecked allocations, and infinite recursion. Handle rays parallel to the ground correctly.
- The result must have recognizable glass and chrome, coherent reflections, visible contact shadows, smooth gradients, and no obvious self-shadow speckles or holes.

TGA output format

Write `out.tga` directly in **uncompressed, 24-bit true-color TGA** format:

- An 18-byte header with image type `2`, no color map, and no image ID.
- Width `1280` and height `720`, stored as little-endian 16-bit values.
- X and Y origins set to zero.
- Pixel depth `24`, no alpha channel.
- Image descriptor `0x20`: top-left origin with left-to-right pixel order.
- Pixels stored in **B, G, R** byte order, row by row from top to bottom.
- No footer or extra metadata. The exact file size must be **2,764,818 bytes**: `18 + 1280 * 720 * 3`.

Open the file in binary mode and write header fields explicitly, avoiding assumptions about struct packing or host endianness. Check file creation, writes, and closing for errors. Exit with status `0` only after successfully producing the complete image; otherwise print a useful error to standard error and return a nonzero status.

Required compile–run–fix workflow

Use the local compiler yourself:

```
cc -std=c89 -pedantic-errors -Wall -Wextra -O2 main.c -lm -o raytracer
./raytracer
```

1. Write the full implementation in `main.c`.
2. Run the compiler command above. Read its diagnostics and correct the source until it builds successfully with no warnings. Keep the strict C89 flags enabled.
3. Run the resulting executable. Correct any crashes, runtime errors, output failures, or rendering defects, then rebuild and rerun as necessary.
4. Verify that the final run exits successfully and produces a newly generated `out.tga` with the required header, dimensions, pixel format, and exact file size. The pixel data must contain a nontrivial rendered image.
5. If an image viewing tool is available, inspect the result for composition, material appearance, orientation, and the fog transition. Fix visible defects and regenerate the image. If visual inspection is unavailable, state that limitation while completing the available file and pixel checks.
6. Finish with a short report listing the artifacts, the actual compile and execution commands used, and the verification performed. Report only checks you actually ran.

Do not stop at a proposed solution, pseudocode, an uncompiled source listing, or a request for the user to compile it. The task is complete when the single-file C89 program has been compiled locally and has successfully rendered the required `out.tga` image.